

BETFRIENDS

KAN-101

Corrigir validacoes e estado local da tela Criar Desafio

Guia de implementacao e validacao para o responsavel da task

Responsavel atual	Guilherme Silva
Epic	KAN-10 - Desafios
Status de referencia	Proximas
Prioridade	High
Tecnologia	Flutter / Dart
Escopo	Correcao local de UI e estado, sem persistencia real de Challenge

1 - Visao geral da task

Objetivo: corrigir inconsistencias locais da tela Criar Desafio sem depender da persistencia real de Challenge. A task deve deixar a interface tecnicamente segura para continuar sendo usada no prototipo, evitando selecoes invalidas, mutacoes incorretas de estado e comunicacao enganosa de funcionalidades que ainda nao existem.

A KAN-101 nao implementa a criacao real de desafios. Ela prepara e corrige a interface atual. A implementacao funcional completa continua pertencendo a KAN-55 e depende de backend, amizade, Match e BetCoins.

1.1 - Identificacao no Jira

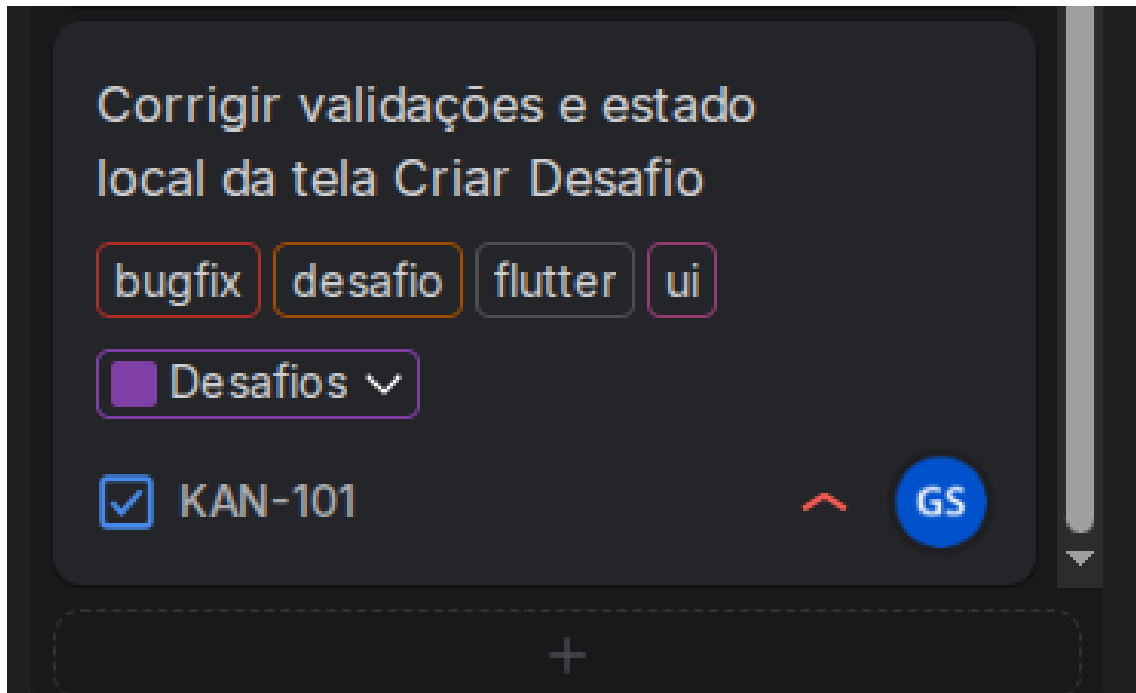


Figura 1 - Card atual da KAN-101 no Jira.

1.2 - Estado atual observado no codigo

- A tela Criar Desafio existe e esta visualmente completa, mas o fluxo ainda e demonstrativo.
- A lista de partidas vem de dados reais, porem a tela permite selecionar partidas que nao deveriam ser elegiveis.
- A selecao inicial de partida e alterada dentro do metodo build, o que cria efeito colateral de estado durante a renderizacao.
- Quando a lista de partidas muda apos um refresh, a selecao atual pode apontar para uma partida que deixou de existir na lista.
- A busca/lista de amigos ainda e mockada e nao pode aparentar ser uma funcionalidade real.
- O botao ENVIAR DESAFIO fecha a tela, mas nao salva Challenge nem executa qualquer persistencia.

2 - Escopo obrigatorio

A task deve entregar os seguintes comportamentos, sem ampliar o escopo para backend ou persistencia.

#	Correcao obrigatoria	Resultado esperado
1	Bloquear partidas inelegiveis	Partidas finalizadas, canceladas ou adiadas nao podem ser escolhidas para um novo desafio.
2	Remover mutacao de estado no build	A selecao inicial nao deve ser alterada diretamente durante a renderizacao.
3	Reconciliar selecao apos refresh	Se a partida selecionada desaparecer ou deixar de ser elegivel, a tela escolhe um estado valido.
4	Preservar valor selecionado	O valor local do desafio nao deve ser perdido sem motivo durante atualizacoes de lista/estado.
5	Nao fingir busca de amigo real	Busca/lista mockada deve ser claramente desabilitada, demonstrativa ou identificada como ind.
6	Nao fingir envio persistido	O botao de envio nao pode comunicar sucesso real enquanto nenhum Challenge e salvo.
7	Manter navegacao atual	O usuario continua conseguindo entrar e sair da tela sem quebrar o fluxo existente.
8	Manter analyzer sem erro bloqueante	A alteracao nao introduz erro de compilacao ou analyzer bloqueante.

2.1 - O que nao deve ser feito nesta task

- Nao criar persistencia real de Challenge.
- Nao implementar Firestore para desafios.
- Nao criar fluxo completo de amizade, solicitacao ou busca real de usuarios.
- Nao implementar reserva de BetCoins.
- Nao implementar maquina de estados PENDING / ACCEPTED / IN_PROGRESS / WAITING_RESULT.
- Nao mover regras sensiveis para o cliente apenas para "fazer funcionar".
- Nao substituir KAN-55. A KAN-101 deve apenas corrigir a UI existente.

Regra pratica: se a alteracao exigir criar uma regra de negocio permanente, persistir dados ou integrar um modulo ainda inexistente, provavelmente esta fora do escopo da KAN-101.

3 - Plano de implementacao

A ordem abaixo reduz risco de regressao e facilita revisar o PR.

3.1 - Mapear estados usados pela tela

- Identificar como a tela recebe PartidasProvider, como monta a lista e onde guarda partidald, valor e amigo selecionado.
- Nao alterar ainda a arquitetura global; primeiro limitar a mudanca ao arquivo/tela e helpers diretamente relacionados.

3.2 - Criar um filtro de partidas elegiveis

- Separar a lista recebida do provider da lista realmente elegivel para novo desafio.
- Obrigatorio: excluir finalizadas, canceladas e adiadas.
- Para partidas ao vivo, nao inventar uma regra nova sem validar a regra de negocio. O teste deve garantir que o comportamento seja explicito e consistente com as regras aprovadas do projeto.

3.3 - Remover mutacao de estado do build

- Nao executar atribuicoes como partidald = partidas.first.id diretamente dentro do build.

- Inicializar/reconciliar selecao em um ponto controlado do ciclo de vida ou apos a atualizacao da lista, usando setState somente quando necessario.

3.4 - Reconciliar selecao quando a lista mudar

- Sempre verificar se partidaId ainda existe entre as partidas elegiveis.
- Se nao existir, selecionar uma opcao valida ou deixar a selecao nula quando nao houver nenhuma partida elegivel.
- Nao manter IDs orfaos apos refresh.

3.5 - Preservar o valor local

- O valor selecionado (10/25/50/100) deve continuar no estado local durante refresh da lista de partidas.
- Mudancas na lista de partidas nao devem redefinir o valor sem uma regra explicita.

3.6 - Corrigir comunicacao de amigos

- Enquanto a busca real nao existir, nao deixar o campo parecer uma busca operacional.
- Opcao segura: desabilitar o campo e exibir texto como "Busca de amigos ainda indisponivel".
- Se os nomes mockados forem mantidos para demonstracao, sinalizar claramente que sao dados demonstrativos.

3.7 - Corrigir o botao ENVIAR DESAFIO

- Nao mostrar mensagem que indique que um desafio foi realmente criado ou enviado.
- Se a navegacao de retorno for mantida para demonstracao, informar explicitamente que nenhum desafio foi salvo.
- Nao adicionar persistencia improvisada nesta task.

4 - Diretriz tecnica para o estado da tela

A implementacao exata pode variar conforme o codigo atual, mas o comportamento deve seguir esta logica.

```
// Pseudocodigo - adaptar aos nomes reais do projeto

List<Partida> elegiveis(List<Partida> todas) {
    return todas.where((p) => p.podeCriarDesafio).toList();
}

void reconciliarSelecao(List<Partida> lista) {
    final atualAindaExiste = lista.any((p) => p.id == partidaId);

    if (!atualAindaExiste) {
        setState(() {
            partidaId = lista.isEmpty ? null : lista.first.id;
        });
    }
}

// Importante:
// - nao chamar setState diretamente como efeito colateral do build
// - nao guardar um id que nao exista mais na lista
// - nao alterar o valor do desafio durante refresh sem motivo
```

O objetivo nao e impor este codigo literalmente. O objetivo e garantir que a renderizacao seja pura e que o estado continue valido quando os dados assincronos mudarem.

4.1 - Estados que a tela precisa distinguir

Estado	Comportamento esperado
Carregando	Mostrar loading sem criar selecao falsa.
Lista com elegiveis	Exibir somente as opcoes permitidas e manter selecao valida.
Lista sem elegiveis	Informar que nao ha partida disponivel para criar desafio.
Erro de partidas	Nao tratar automaticamente como lista vazia se o provider disponibiliza um erro. Alinhar com a KAN-100 para evita
Refresh	Revalidar selecao e preservar o restante do estado local.

4.2 - Cuidado com conflito com a KAN-100

A KAN-100 foi reaberta porque ainda precisa deixar o tratamento de erro da Home coerente com Criar Desafio. Antes de alterar o mesmo bloco de UI, conferir a branch/PR da KAN-100. Se ambos mexerem no mesmo trecho, combinar a ordem de merge para evitar conflito e retrabalho.

5 - Testes e validacao

Antes do PR, validar manualmente pelo menos os cenarios abaixo.

Cenário	Resultado esperado
Partida futura valida	Pode ser selecionada normalmente.
Partida finalizada	Nao aparece como opcao elegivel ou nao pode ser selecionada.
Partida cancelada	Nao pode ser selecionada.
Partida adiada	Nao pode ser selecionada.
Partida ao vivo	Comportamento testado e coerente com a regra de negocio vigente; nao deixar comportament

Cenario	Resultado esperado
Refresh remove a partida selecionada	A selecao muda para uma opcao valida ou fica vazia sem ID invalido.
Refresh mantem a partida selecionada	A selecao permanece, sem piscar/redefinir desnecessariamente.
Mudanca de partida	Valor 10/25/50/100 permanece consistente.
Busca de amigo	Nao aparenta executar uma busca real quando o backend nao existe.
ENVIAR DESAFIO	Nao informa sucesso persistido. O retorno de navegacao continua funcionando.
Sem partidas elegiveis	Tela apresenta estado claro e nao quebra.

Comandos minimos antes do PR:

```
flutter pub get
flutter analyze
flutter test
```

6 - Fluxo Git e Jira

Para manter rastreabilidade, usar a chave da issue em branch, commits e Pull Request.

Branch sugerida:

feature/KAN-101-corrigir-criar-desafio

Commit sugerido:

KAN-101: corrige validacoes e estado local da tela Criar Desafio

PR sugerido:

KAN-101 - Corrigir validacoes e estado local da tela Criar Desafio

Destino do PR:

qas

6.1 - Status no Jira

Momento	Status
Antes de iniciar	Proximas
Quando comecar o codigo	Em andamento
Se surgir bloqueio real externo	Aguardando dependencias
Apos merge em qas + validacao dos criterios	Concluido

7 - Checklist de conclusao

Partidas finalizadas nao podem ser selecionadas.

Partidas canceladas nao podem ser selecionadas.

Partidas adiadas nao podem ser selecionadas.

Nao existe mutacao de partidald dentro do build.

Selecao e corrigida quando a lista muda.

Valor selecionado permanece consistente.

Busca de amigo nao aparenta ser funcional quando nao e.

ENVIAR DESAFIO nao comunica persistencia que nao existe.

Navegacao da tela continua funcionando.

flutter analyze sem erro bloqueante.

flutter test passando.

Branch, commit e PR usam KAN-101.

Codigo foi mergeado em qas.

Critérios de aceite foram conferidos apos o merge.

Definicao de pronto

A KAN-101 pode ser marcada como Concluida somente quando a tela nao oferecer selecoes invalidas, nao modificar estado de forma insegura durante o build, nao simular funcionalidades inexistentes como se fossem reais, mantiver o fluxo de navegacao e tiver sido validada na branch qas.

8 - Resultado esperado para o responsavel

Ao finalizar esta task, a tela Criar Desafio ainda sera uma interface preparatoria - nao um desafio real persistido. A diferenca e que ela deixara de induzir o usuario a erros ou apresentar comportamentos tecnicamente inconsistentes. Isso permite que a futura KAN-55 conecte a mesma interface ao backend sem precisar refazer a base local de estado.